

CS1003

Programming in C

Module - 4

Content of Module-4

- **Chapter-1: Arrays**
 - Declaration, initialization, indexing, array size
 - One-dimensional, Two-dimensional and multi-dimensional arrays
 - Array traversal and manipulation
 - Searching- Linear search, Binary search
 - Sorting – Selection sort, Bubble sort, Insertion sort
- **Chapter-2: Strings**
 - Strings as array of Characters, String Library functions

- An array is a collection of elements of the same type that are referenced by a common name.
- **Fixed Size:** Arrays have a fixed size determined at declaration.
- **Zero-Based Index:** Array elements are accessed using indices starting from 0.
- **Homogeneous:** Arrays store elements of the same data type.
 - Compared to the basic data type (`int`, `float` & `char`) it is an aggregate or derived data type.
 - All the elements of an array occupy a set of contiguous memory locations.
 - Why need to use array type?
 - Consider the following issue:

"We have a list of 1000 students' marks of an integer type. If using the basic data type (`int`), we will declare something like the following.."

```
int studMark0, studMark1, studMark2, ..., studMark999;
```

- Can you imagine how long we have to write the declaration part by using normal variable declaration?

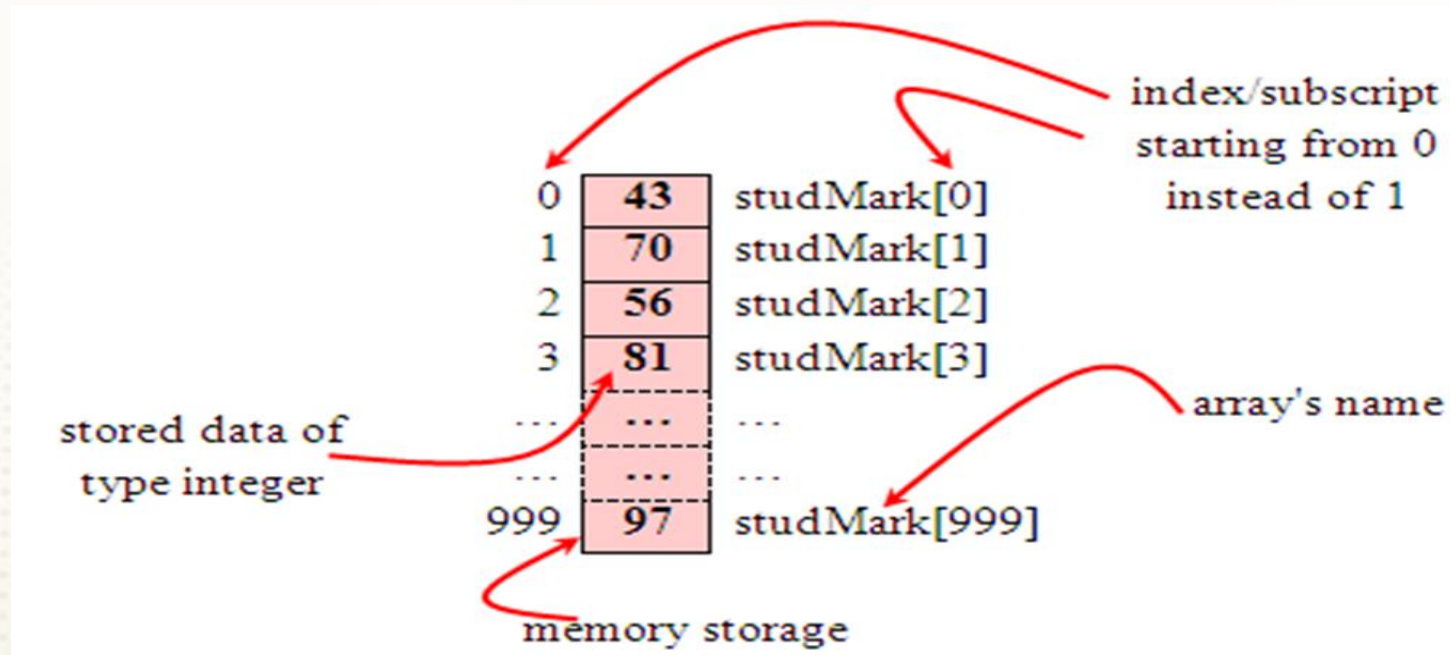
```
int main(void)
{
    int studMark1, studMark2, studMark3,
    studMark4, ..., ..., studMark998, stuMark999,
    studMark1000;
    ...
    ...
    return 0;
}
```

ARRAYS

- By using an array, we just declare like this,

```
int studMark[1000];
```

- This will reserve 1000 contiguous memory locations for storing the students' marks.
- Graphically, this can be depicted as in the following figure.



- This absolutely has simplified our declaration of the variables.
- We can use index or subscript to identify each element or location in the memory.
- Hence, if we have an index of `jIndex`, `studMark[jIndex]` would refer to the *jIndex*th element in the array of `studMark`.
- For example, `studMark[0]` will refer to the first element of the array.
- Thus by changing the value of `jIndex`, we could refer to any element in the array.
- So, array has simplified our declaration and of course, manipulation of the data.

One-dimensional arrays

- Dimension refers to the array's size, which is how big the array is.
- A single or one dimensional array declaration has the following form,

```
array_element_data_type array_name[array_size];
```

- Here, *array_element_data_type* define the base type of the array, which is the type of each element in the array.
- *array_name* is any valid C / C++ identifier name that obeys the same rule for the identifier naming.
- *array_size* defines how many elements the array will hold.

One-dimensional arrays

- For example, to declare an array of 30 characters, that construct a people name, we could declare,

```
char    cName[30];
```

- Which can be depicted as follows,
- In this statement, the array character can store up to 30 characters with the first character occupying location `cName[0]` and the last character occupying `cName[29]`.
- Note that the index runs from 0 to 29. In C, an index always starts from 0 and ends with array's (size-1).
- So, take note the difference between the array size and subscript/index terms.

J	cName[0]
o	cName[1]
d	cName[2]
i	cName[3]
e	cName[4]
...	cName[5]
...	...
...	...
r	cName[29]

One-dimensional arrays

- Examples of the one-dimensional array declarations,

```
int      xNum[20], yNum[50];  
float    fPrice[10], fYield;  
char     chLetter[70];
```

- The first example declares two arrays named `xNum` and `yNum` of type `int`. Array `xNum` can store up to 20 integer numbers while `yNum` can store up to 50 numbers.
- The second line declares the array `fPrice` of type `float`. It can store up to 10 floating-point values.
- `fYield` is basic variable which shows array type can be declared together with basic type provided the type is similar.
- The third line declares the array `chLetter` of type `char`. It can store a string up to 69 characters.
- Why 69 instead of 70? Remember, a string has a null terminating character (`\0`) at the end, so we must reserve for it.

One-dimensional arrays- Initialization

- Initializing all specified memory locations
- Partial array initialization
- Initialization without size
- String initialization
- Assigning values to array

One-dimensional arrays- Initialization

- An array may be initialized at the time of declaration.
- Giving initial values to an array.
- Initialization of an array may take the following form,

```
type    array_name[size] = {a_list_of_value};
```
- For example:

```
int      idNum[7] = {1, 2, 3, 4, 5, 6, 7};  
float    fFloatNum[5] = {5.6, 5.7, 5.8, 5.9, 6.1};  
char     chVowel[6] = {'a', 'e', 'i', 'o', 'u', '\0'};
```
- The first line declares an integer array `idNum` and it immediately assigns the values 1, 2, 3, ..., 7 to `idNum[0]`, `idNum[1]`, `idNum[2]`, ..., `idNum[6]` respectively.
- The second line assigns the values 5.6 to `fFloatNum[0]`, 5.7 to `fFloatNum[1]`, and so on.
- Similarly the third line assigns the characters 'a' to `chVowel[0]`, 'e' to `chVowel[1]`, and so on. Note again, for characters we must use the single apostrophe/quote (') to enclose them.
- Also, the last character in `chVowel` is NULL character ('\0').

- Initialization of an array of type char for holding strings may take the following form,

```
char    array_name[size] = "string_lateral_constant";
```

- For example, the array chVowel in the previous example could have been written more compactly as follows,

```
char    chVowel[6] = "aeiou";
```

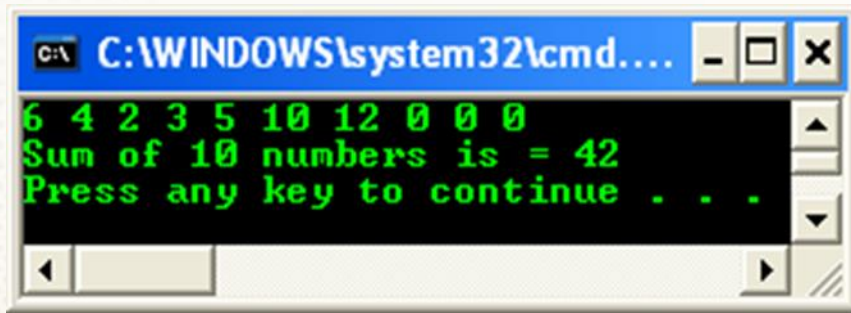
- When the value assigned to a character array is a string (which must be enclosed in double quotes), the compiler automatically supplies the NULL character but we still have to reserve one extra place for the NULL.
- For unsized array (variable sized), we can declare as follow,

```
char chName[ ] = "Mr. Dracula";
```

- C compiler automatically creates an array which is big enough to hold all the initializer.

Examples: One-dimensional arrays

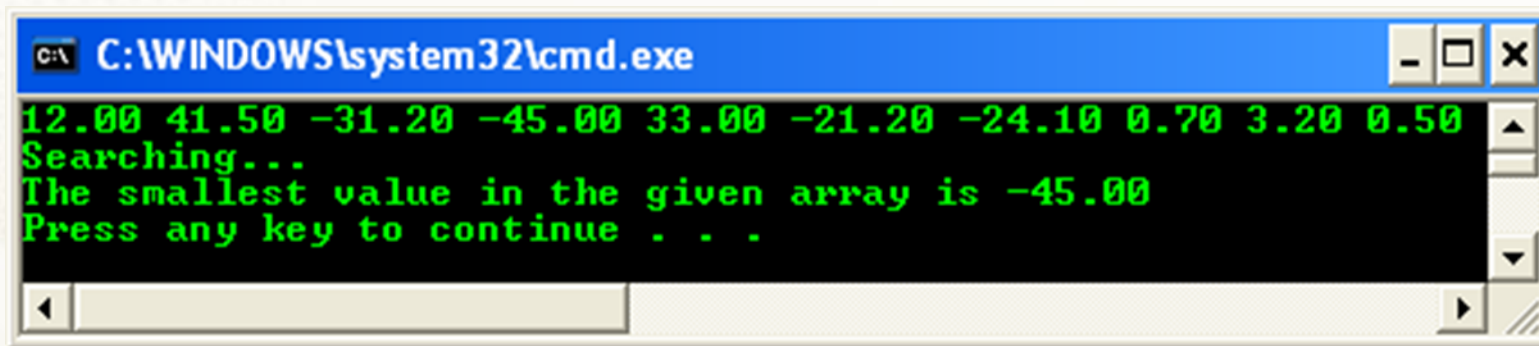
- Arrays allow programmers to group related items of the same data type in one variable.
- However, when referring to an array, one has to specify not only the array or variable name but also the index number of interest.
- [Program example 1: Sum of array's element.](#)
- Notice the array's element which is not initialized is set to 0 automatically.



```
C:\WINDOWS\system32\cmd....  
6 4 2 3 5 10 12 0 0 0  
Sum of 10 numbers is = 42  
Press any key to continue . . .
```

Examples: One-dimensional arrays

- Program example 2: Searching the smallest value.
- Finding the smallest element in the array named `fSmallest`.
- First, it assumes that the smallest value is in `fSmallest[0]` and assigns it to the variable `nSmall`.
- Then it compares `nSmall` with the rest of the values in `fSmallest`, one at a time.
- When an element is smaller than the current value contained in `nSmall`, it is assigned to `nSmall`. The process finally places the smallest array element in `nSmall`.



```
C:\WINDOWS\system32\cmd.exe
12.00 41.50 -31.20 -45.00 33.00 -21.20 -24.10 0.70 3.20 0.50
Searching...
The smallest value in the given array is -45.00
Press any key to continue . . .
```

Examples: One-dimensional arrays

- Program example 3: Searching the biggest value. By modifying the previous example we can search the biggest value.
- Program example 4: Searching the location for the given value in an array
- Program example 5: Storing and reading a string
- Program example 6: Storing and reading array content and its index

- **C Program to Find Sum of Array Elements using Loops and Variables**

- #include <stdio.h>

```
int main()
```

```
{
```

```
    int size;
```

```
    printf("Enter size of the array: ");
```

```
    scanf("%d",&size);
```

```
    //Declaring array
```

```
    int arr[size];
```

```
    printf("Enter array elements\n");
```

```
    // Input array elements
```

```
    for(int i=0;i<size;i++)
```

```
        scanf("%d",&arr[i]);
```

```
    int sum=0;
```

```
        // Loop to find sum
```

```
    for(int i=0;i<size;i++)
```

```
        sum+=arr[i];
```

```
        //Print sum
```

```
    printf("Sum of the array is: %d",sum);
```

```
    return 0;
```

```
}
```

Two-dimensional Arrays

- A two dimensional array has two subscripts/indexes.
- The first subscript refers to the row, and the second, to the column.
- Its declaration has the following form,

```
data_type    array_name[1st dimension size][2nd dimension size];
```

- For examples,

```
int          xInteger[3][4];  
float        matrixNum[20][25];
```

- The first line declares `xInteger` as an integer array with 3 rows and 4 columns.
- Second line declares a `matrixNum` as a floating-point array with 20 rows and 25 columns.

Two-dimensional Arrays

- If we assign initial string values for the 2D array it will look something like the following,

```
char Name[6][10] = {"Mr. Bean", "Mr. Bush", "Nicole",  
                    "Kidman", "Arnold", "Jodie"};
```

- Here, we can initialize the array with 6 strings, each with maximum 9 characters long.
- If depicted in rows and columns it will look something like the following and can be considered as contiguous arrangement in the memory.

Second index

Columns

row

0 1 2 3 4 5 6 7 8 9

Name[2][2]

First index

Name[row][column] = Name[0][0]

0	M	r	.		B	e	a	n	\0	
1	M	r	.		B	u	s	h	\0	
2	N	i	c	o	l	e	\0			
3	K	i	d	m	a	n	\0			
4	A	r	n	o	l	d	\0			
5	J	o	d	i	e	\0				

Name[5][4]

Name[3][5]

Two-dimensional Arrays

- Take note that for strings the null character (`\0`) still needed.
- From the shaded square area of the figure we can determine the size of the array.
- For an array `Name [6] [10]`, the array size is $6 \times 10 = 60$ and equal to the number of the colored square. In general, for

`array_name[x][y];`

- The array size is = First index $\times$ second index = xy .
- This also true for other array dimension, for example three dimensional array,

`array_name[x][y][z];` $\Rightarrow$ First index $\times$ second index $\times$ third index = xyz

- For example,

`ThreeDimArray[2][4][7]` = $2 \times 4 \times 7 = 56$.

- And if you want to illustrate the 3D array, it could be a cube with wide, long and height dimensions.

Two-dimensional Arrays

2-D Array declaration and initialization

```
int matrix[2][3] = { {1, 4, 2}, {3, 6, 8} };
```

The first dimension represents the number of rows [2], while the second dimension represents the number of columns [3]. The values are placed in row-order, and can be visualized like this:

Access the Elements of a 2D Array

To access an element of a two-dimensional array, you must specify the index number of both the row and column.

This statement accesses the value of the element in the **first row (0)** and **third column (2)** of the **matrix** array.

Example

```
int matrix[2][3] = { {1, 4, 2}, {3, 6, 8} };
```

```
printf("%d", matrix[0][2]); // Outputs 2
```

Two-dimensional Arrays

Change Elements in a 2D Array

To change the value of an element, refer to the index number of the element in each of the dimensions:

The following example will change the value of the element in the **first row (0)** and **first column (0)**:

Example

```
int matrix[2][3] = { {1, 4, 2}, {3, 6, 8} };
```

```
matrix[0][0] = 9;
```

```
printf("%d", matrix[0][0]); // Now outputs 9 instead of 1
```

Two-dimensional Arrays

Loop Through a 2D Array

To loop through a multi-dimensional array, you need one loop for each of the array's dimensions.

The following example outputs all elements in the **matrix** array:

Example

```
int matrix[2][3] = { {1, 4, 2}, {3, 6, 8} };
```

```
int i, j;
for (i = 0; i < 2; i++) {
    for (j = 0; j < 3; j++) {
        printf("%d\n", matrix[i][j]);
    }
}
```

Two-dimensional Arrays

- Examples: 1. WAP to find the addition of two matrices
- 2. WAP to find perform the multiplication of two matrices

```
#include <stdio.h>

int main() {
    int m, n, i, j;

    printf("Enter the
    number of rows and
    columns of the
    matrices: ");

    scanf("%d%d", &m,
    &n);

    int a[m][n], b[m][n],
    c[m][n];

    printf("Enter the
    elements of matrix A:
    \n");
```

```
    for (i = 0; i < m; i++) {
        for (j = 0; j < n; j++) {
            scanf("%d", &a[i][j]);
        }
    }

    printf("Enter the
    elements of matrix B:
    \n");

    for (i = 0; i < m; i++) {
        for (j = 0; j < n; j++) {
            scanf("%d", &b[i][j]);
        }
    }
```

```
// add the matrices
for (i = 0; i < m; i++) {
    for (j = 0; j < n; j++) {
        c[i][j] = a[i][j] + b[i][j];
    }
}

// print the result
printf("The sum of the
two matrices is: \n");

for (i = 0; i < m; i++) {
    for (j = 0; j < n; j++) {
```

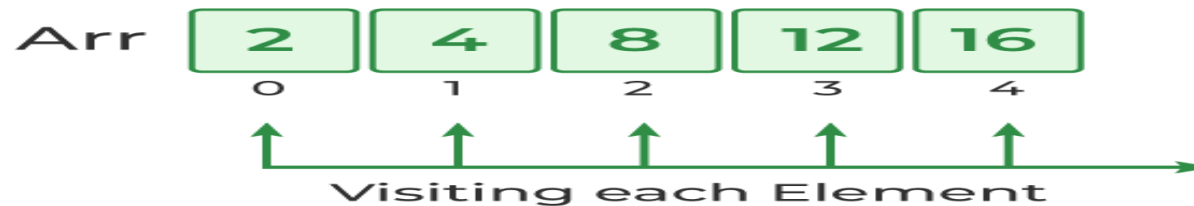
Array Traversal

- C Array Traversal
- Traversal is the process in which we visit every element of the data structure. For C array traversal, we use loops to iterate through each element of the array.
- Array Traversal using for Loop

```
for (int i = 0; i < N; i++) {  
    array_name[i];  
}
```

Array Transversal

```
for ( int i = 0; i < Size; i++){  
    arr[i];  
}
```



Array Traversal

// C Program to demonstrate the use of array

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    // array declaration and initialization
```

```
    int arr[5] = { 10, 20, 30, 40, 50 };
```

```
    arr[2] = 100;           // modifying element at index 2
```

```
    // traversing array using for loop
```

```
    printf("Elements in Array: ");
```

```
    for (int i = 0; i < 5; i++) {
```

```
        printf("%d ", arr[i]);
```

```
    }
```

```
    return 0;
```

```
}
```

Array Manipulation-Insertion and Deletion

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int a[100];
    int element,i,loc,size,n=0,j=0;
    printf("Enter the size of an array\n");
    scanf("%d",&size);
    printf("Enter %d array elements\n",size);
    for(i=0;i<size;i++)
    {
        scanf("%d",&a[i]);
    }
    printf("List before Insertion: ");
    for(i=0;i<size;i++)
    {
        printf("%d ",a[i]);
    }
}
```

Array Manipulation-Insertion and Deletion

```
printf("\nEnter an element to insert\n");
scanf("%d",&element);
printf("Enter a position to insert an element %d\n",element);
scanf("%d",&loc);
loc--;
for(i=size-1;i>=loc;i--)
{
    a[i+1]=a[i];
}
a[loc]=element;
printf("\nList after Insertion: ");
for(i=0;i<size+1;i++)
{
    printf("%d ",a[i]);
}
for(i=0;i<size+1;i++)
{
    printf("%d ",a[i]);
}
```

Array Manipulation-Insertion and Deletion

```
printf("\nEnter an element to delete\n");
scanf("%d",&n);
i=0;
for(i=0;i<size;i++)
{
    if(a[i]==n)
    {
        for(j=i;j<(size-1);j++)
        {
            a[j]=a[j+1];
        }
        break;
    }
}
printf("List after deletion\n");
for(i=0;i<(size-1);i++)
{
    printf("%d ",a[i]);
}
return 0;
}
```

Array Manipulation- Assigning values in an array in C

We initialize the array after the declaration by assigning the initial value to each element individually.

Example:

```
int a[5]; //declaration of an array  
a[2]=10;  
a[0]=3;
```

We can use for loop, while loop, or do-while loop to assign the value to each element of the array.

```
for(i=0;i<n;i++)  
{   scanf("%d",&a[i]);  
}
```

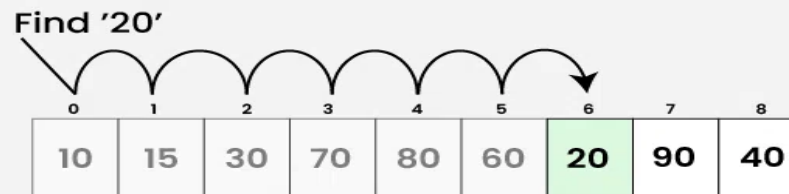
Array Manipulation- Swapping values in an array in C

```
#include<stdio.h>
void main()
{
    int a[ ]={1,2,3,4,5},i, temp,n=5;
    printf("\n array before interchange");
    for(i=0;i<n;i++)
        printf("\t%d",a[i]);
    for(i=0;i<n-1;i+=2)
    {
        temp=a[i];
        a[i]=a[i+1];
        a[i+1]=temp;
    }
```

```
printf("\n array after interchange");
for(i=0;i<n;i++)
    printf("\t%d",a[i]);
```

- Linear Search Algorithm
- The linear search algorithm is defined as a sequential search algorithm that starts at one end and goes through each element of a list until the desired element is found; otherwise, the search continues till the end of the dataset.

Linear Search Algorithm



```
#include <stdio.h>

int main()
{
    int array[100], key, i, n;

    printf("Enter number of elements in  
array\n");
    scanf("%d", &n);

    printf("Enter %d integer(s)\n", n);

    for (i = 0; i < n; i++)
        scanf("%d", &array[i]);

    printf("Enter a number to search\n");
    scanf("%d", &key);
```

```
    for (i = 0; i < n; i++)
    {
        if (array[i] == search)    /* If  
required element is found */
            return i;
    }
    return -1;    //unsuccessful search
}
```

Removing Duplicates in an Array

```
#include <stdio.h>

int main()
{
    int n, count = 0;

    printf("Enter number of elements in the array: ");

    scanf("%d", &n);

    int arr[n], temp[n];

    if(n==0)
    {
        printf("No element inside the array.");
        exit(0);
    }

    printf("Enter elements in the array: ");
    for (int i = 0; i < n; i++)
    {
```

```
        scanf("%d", &arr[i]);
    }
    printf("\nArray Before Removing Duplicates: ");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    // To store unique elements in temp after
    removing the duplicate elements
    for (int i = 0; i < n; i++)
    {
        int j;
        for (j = 0; j < count; j++)
        {
            if (arr[i] == temp[j])
                break;
        }
        if (j == count)
        {
            temp[count] = arr[i];
            count++;
        }
    }
```

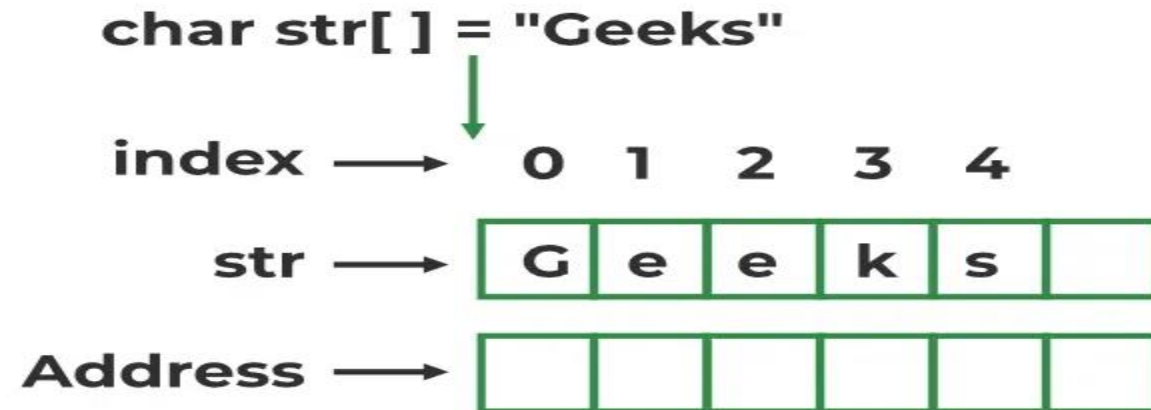
```
    printf("\nArray After Removing Duplicates: ");
    for (int i = 0; i < count; i++)
        printf("%d ", temp[i]);

    return 0;
}
```

String

- A String in C programming is a sequence of characters terminated with a null character '\0'. The C String is stored as an array of characters. The difference between a character array and a C string is that the string in C is terminated with a unique character '\0'.

String in C



strcat() function in C is used for string concatenation. It will append a copy of the source string to the end of the destination string.

- strncat()

This function is used for string handling. This function appends not more than n characters from the string pointed to by src to the end of the string pointed to by dest plus a terminating Null-character. strlen() Function

The strlen() function calculates the length of a given string. It doesn't count the null character '\0'. strcmp() Function

The `strcmp()` is a built-in library function in C. This function takes two strings as arguments and compares these two strings lexicographically.

Parameters

`str1`: This is the first string to be compared.

`str2`: This is the second string to be compared.

Return Value

If `str1` is less than `str2`, the return value is less than 0.

If `str1` is greater than `str2`, the return value is greater than 0.

If `str1` is equal to `str2`, the return value is 0.

`strncmp()`

This function lexicographically compares the first `n` characters from the two null-terminated strings and returns an integer based on the outcome.

`strncmp()`

This function lexicographically compares the first *n* characters from the two null-terminated strings and returns an integer based on the outcome.

`strcpy`

The `strcpy()` is a standard library function in C and is used to copy one string to another. In C, it is present in `<string.h>` header file.

`strncpy()`

The function `strncpy()` is similar to `strcpy()` function, except that at most *n* bytes of *src* are copied.

Where *n* is the first *n* characters to be copied from *src* to *dest*.

`strchr()`

The `strchr()` function in C is a predefined function used for string handling. This function is used to find the first occurrence of a character in a string.

- `strstr()`
- The `strstr()` function in C is used to search the first occurrence of a substring in another string.

Thank You